

APPLICATIONS:

2) Build a bot that assists customer support agents by fetching relevant information from databases or knowledge bases

Aim

To develop a UiPath Customer Support Bot that helps customer support agents by retrieving customer information from a database (or DataTable) using a Customer ID and displaying the customer's details such as name, email, order ID, amount, and order status.

Algorithm

STEP 1: Start the process.

STEP 2: Read the customer data from the database (or Excel/DataTable).

STEP 3: Display an Input Dialog and ask the user to enter the Customer ID.

STEP 4: Search the DataTable for the entered Customer ID.

STEP 5: Check whether the customer record is found.

STEP 6: If the customer exists, retrieve the customer details such as:

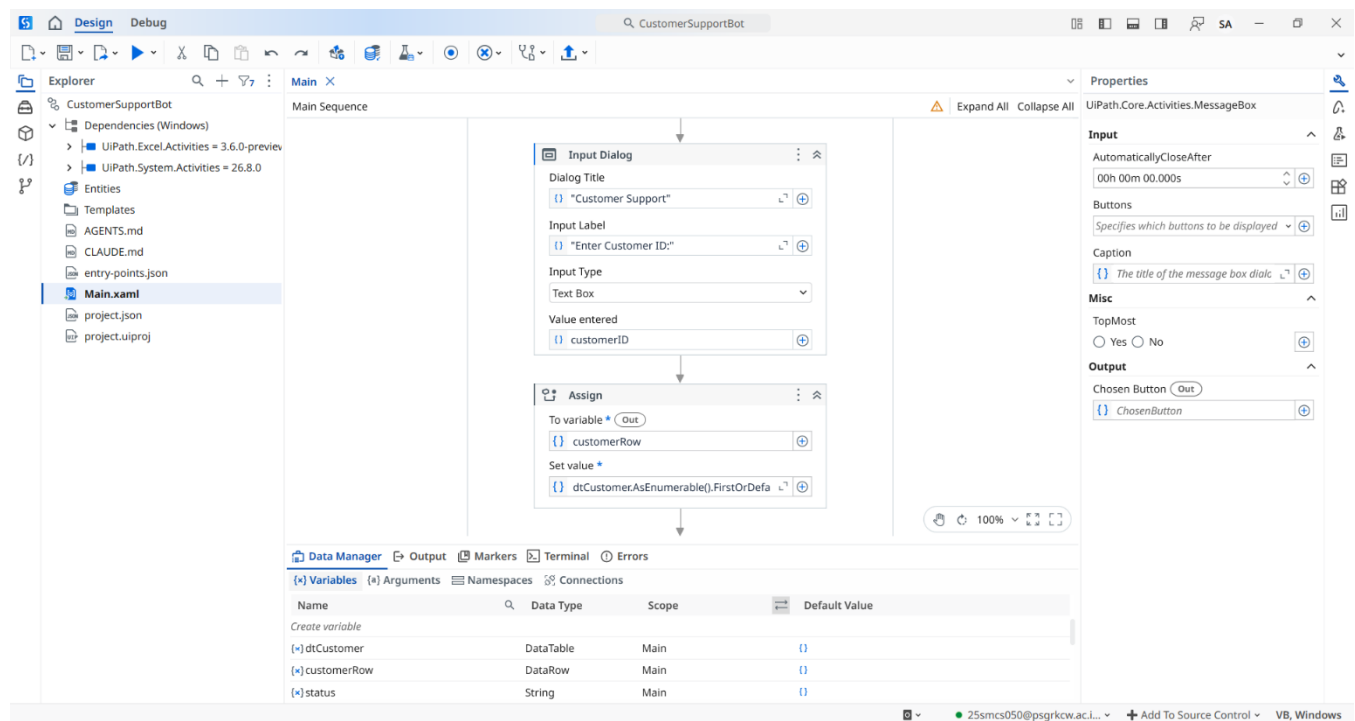
- Customer Name
- Email
- Order ID
- Total Amount
- Order Status

STEP 7: Display the retrieved customer information in a Message Box.

STEP 8: If the Customer ID is not found, display the message "Customer not found. Please check the Customer ID."

STEP 9: End the process.

WORKFLOW



CustomerSupportBot

Dependencies (Windows)

- UIPath.Excel.Activities = 3.6.0-preview
- UIPath.System.Activities = 26.8.0

Entities

Templates

AGENTS.md

CLAUDE.md

entry-points.json

Main.xaml

project.json

project.uiproj

Main Sequence

Input Dialog

Dialog Title: "Customer Support"

Input Label: "Enter Customer ID:"

Input Type: Text Box

Value entered: customerID

Assign

To variable: Out

customerRow

Set value: dtCustomer.AsEnumerable().FirstOrDefault()

Properties

UIPath.Core.Activities.MessageBox

Input

AutomaticallyCloseAfter: 00h 00m 00.000s

Buttons: Specifies which buttons to be displayed

Caption: The title of the message box dialog

Misc

TopMost: Yes No

Output

Chosen Button: Out

ChosenButton

Data Manager

Output

Markers

Terminal

Errors

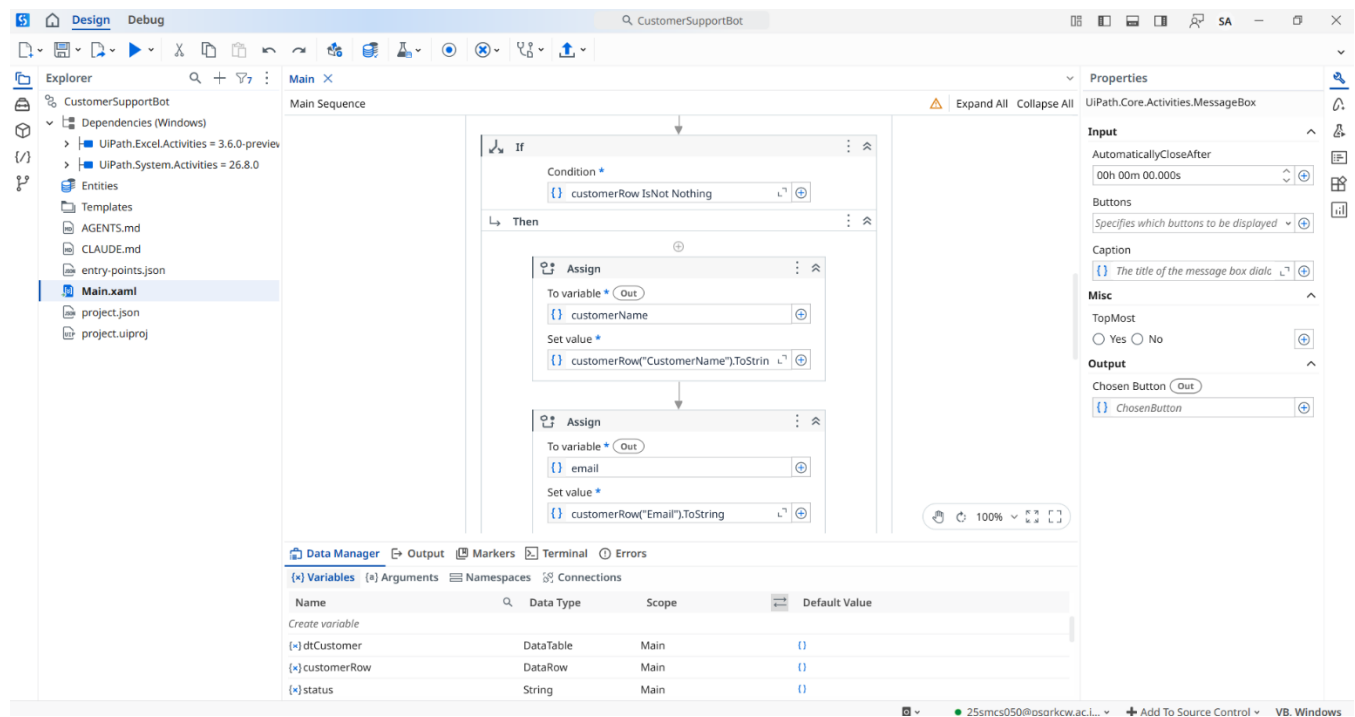
Variables

Arguments

Namespaces

Connections

Name	Data Type	Scope	Default Value
dtCustomer	DataTable	Main	
customerRow	DataRow	Main	
status	String	Main	



CustomerSupportBot

Dependencies (Windows)

- UIPath.Excel.Activities = 3.6.0-preview
- UIPath.System.Activities = 26.8.0

Entities

Templates

AGENTS.md

CLAUDE.md

entry-points.json

Main.xaml

project.json

project.uiproj

Main Sequence

If

Condition: customerRow IsNot Nothing

Then

Assign

To variable: Out

customerName

Set value: customerRow["CustomerName"].ToString()

Assign

To variable: Out

email

Set value: customerRow["Email"].ToString()

Properties

UIPath.Core.Activities.MessageBox

Input

AutomaticallyCloseAfter: 00h 00m 00.000s

Buttons: Specifies which buttons to be displayed

Caption: The title of the message box dialog

Misc

TopMost: Yes No

Output

Chosen Button: Out

ChosenButton

Data Manager

Output

Markers

Terminal

Errors

Variables

Arguments

Namespaces

Connections

Name	Data Type	Scope	Default Value
dtCustomer	DataTable	Main	
customerRow	DataRow	Main	
status	String	Main	

Design Debug CustomerSupportBot

Explorer

- CustomerSupportBot
 - Dependencies (Windows)
 - UIPath.Excel.Activities = 3.6.0-preview
 - UIPath.System.Activities = 26.8.0
 - Entities
 - Templates
 - AGENTS.md
 - CLAUDE.md
 - entry-points.json
 - Main.xaml**
 - project.json
 - project.uiproj

Main Sequence

Assign

To variable * Out

{ } orderID

Set value *

{ } customerRow("OrderID").ToString

Assign

To variable * Out

{ } amount

Set value *

{ } Convert.ToDouble(customerRow("Amount").ToString)

Assign

To variable * Out

{ } status

Set value *

{ } customerRow("Status").ToString

Properties

UIPath.Core.Activities.MessageBox

Input

AutomaticallyCloseAfter

00h 00m 00.000s

Buttons

Specifies which buttons to be displayed

Caption

{ } The title of the message box dialog

Misc

TopMost

Yes No

Output

Chosen Button Out

{ } ChosenButton

Data Manager

Variables Arguments Namespaces Connections

Name	Data Type	Scope	Default Value
Create variable			
(*) dtCustomer	DataTable	Main	{ }
(*) customerRow	DataRow	Main	{ }
(*) status	String	Main	{ }

25smcs050@osarkw.ac.i... Add To Source Control VB, Windows

Design Debug CustomerSupportBot

Explorer

- CustomerSupportBot
 - Dependencies (Windows)
 - UIPath.Excel.Activities = 3.6.0-preview
 - UIPath.System.Activities = 26.8.0
 - Entities
 - Templates
 - AGENTS.md
 - CLAUDE.md
 - entry-points.json
 - Main.xaml**
 - project.json
 - project.uiproj

Main Sequence

Set value *

{ } status

{ } customerRow("Status").ToString

Message Box

Text *

{ } "Customer Name: " + customerName + E

Else

Message Box

Text *

{ } "Customer not found. Please check the C

Properties

UIPath.Core.Activities.MessageBox

Input

AutomaticallyCloseAfter

00h 00m 00.000s

Buttons

Specifies which buttons to be displayed

Caption

{ } The title of the message box dialog

Misc

TopMost

Yes No

Type: Sequence

Name: Do

Executes a set of child activities according to a single, defined order.

Included in Default Activities

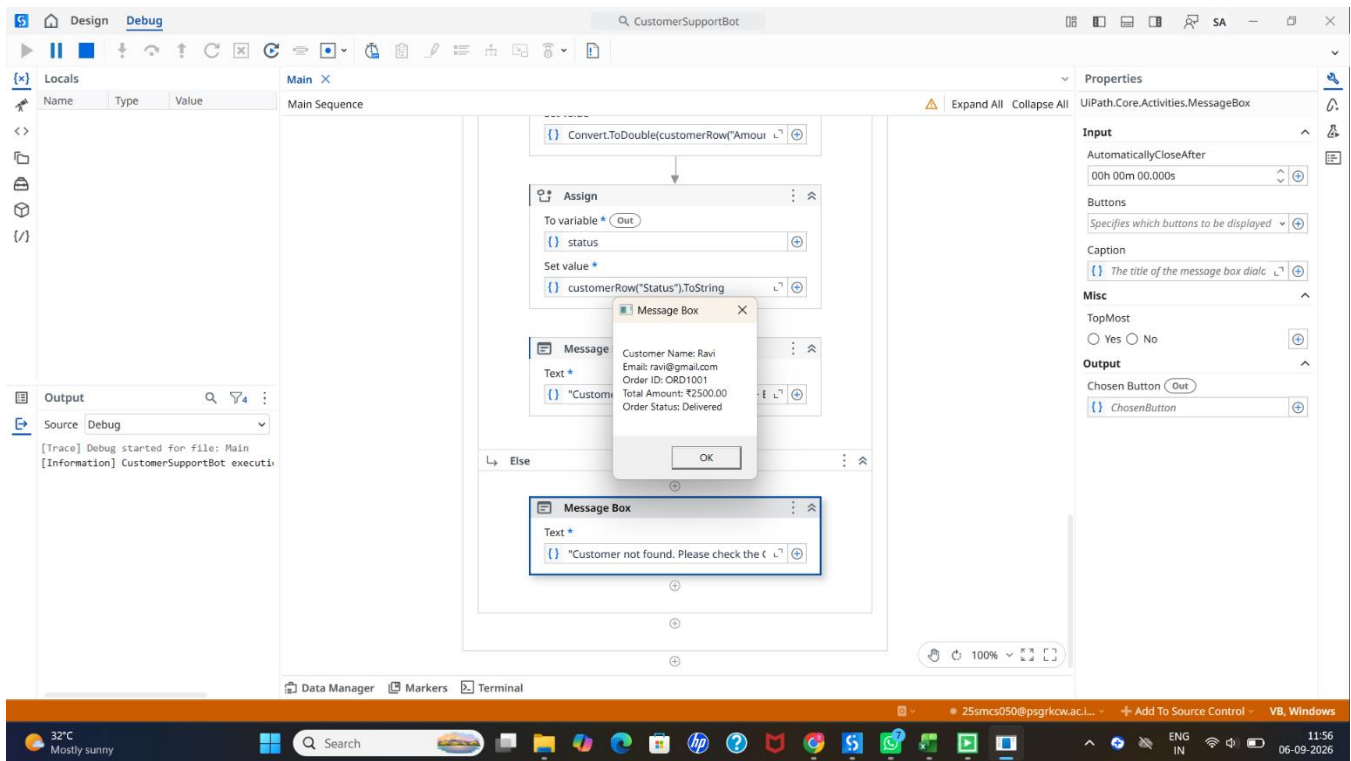
Data Manager

Variables Arguments Namespaces Connections

Name	Data Type	Scope	Default Value
Create variable			
(*) dtCustomer	DataTable	Main	{ }
(*) customerRow	DataRow	Main	{ }
(*) status	String	Main	{ }

25smcs050@psgrkcw.ac.i... Add To Source Control VB, Windows

OUTPUT



RESULT:

Thus, the program has been executed successfully and output is verified.

3) Design Captcha automation in uipath

Aim

To design a simple CAPTCHA verification automation in UiPath that generates a random CAPTCHA, accepts the user's input, and displays whether the entered CAPTCHA is correct or incorrect.

Algorithm

1. Start the UiPath workflow.
2. Generate a random number using the Random function.
3. Store the random number in a variable called randomno.
4. Display an Input Dialog showing the CAPTCHA value.
5. Store the user's entered CAPTCHA in the variable Captured.
6. Use an If condition to compare Captured with randomno.
7. If both values are equal, display "Captcha is correct".
8. Otherwise, display "Captcha is incorrect".
9. End the workflow.

WORKFLOW

The screenshot displays the UiPath Studio interface for a workflow named "BlankProcess1". The workflow is designed in the "Main Sequence" and consists of three main activities:

- Assign:** A variable named "randomno" is assigned the value of "New Random().Next(0, 10000)".
- Input Dialog:** A dialog box titled "Captcha Entry" is shown. The input label is "Enter the Captcha" & randomno.ToString(). The input type is "Text Box" and the value entered is "CapEntered".
- If:** A conditional activity with the condition "CapEntered=Randomno".

The Properties panel on the right shows the properties for the "Input Dialog" activity. The "Label" property is set to "Enter the Captcha" & randomno.ToString(). The "Title" property is set to "Captcha".

The Data Manager table at the bottom shows the following data:

Name	Data Type	Scope	Default Value
randomno	Int32	Main	()
CapEntered	Int32	Main	()

Design Debug

BlankProcess1

Explorer

- BlankProcess1
 - Dependencies (Windows)
 - UIPath.System.Activities = 26.6.3
 - UIPath.UIAutomation.Activities = 26.6.3
 - Entities
 - Templates
 - AGENTS.md
 - CLAUDE.md
 - entry-points.json
 - Main.xaml
 - project.json
 - project.lproj

Main Sequence

Input Type

Text Box

Value entered

CapEntered

If

Condition *

CapEntered=Randomno

Then

Message Box

Text *

Capcha is correct

Else

Message Box

Text *

Capcha is incorrect

Data Manager

Output

Markers

Terminal

Errors

Variables

Arguments

Namespaces

Connections

Name	Data Type	Scope	Default Value
CapEntered	Int32	Main	()
randomno	Int32	Main	()

Properties

Argument 'Result': BC30451: 'CapEntered' is not declared. It m...

System.Activities.Statements.Sequence

Common

DisplayName

Main Sequence

Misc

Private

Design Debug

BlankProcess1

Locals

Name	Type	Value
CapEntered	Int32	10000
randomno	Int32	10000

Main Sequence

Assign

To variable *

Out

randomno

Set value *

New Random().Next(0, 10000)

Input Dialog

Dialog Title

Capcha Entry

Input Label

Capcha Entry

Enter the Capcha 5054

OK

If

Condition *

CapEntered=Randomno

Then

Message Box

Text *

Capcha is correct

Else

Message Box

Text *

Capcha is incorrect

Call Stack

Activity Name

Activity Type

File Path

Data Manager

Markers

Terminal

Properties

One or more children have validation errors or warnings.

System.Activities.Statements.Sequence

Common

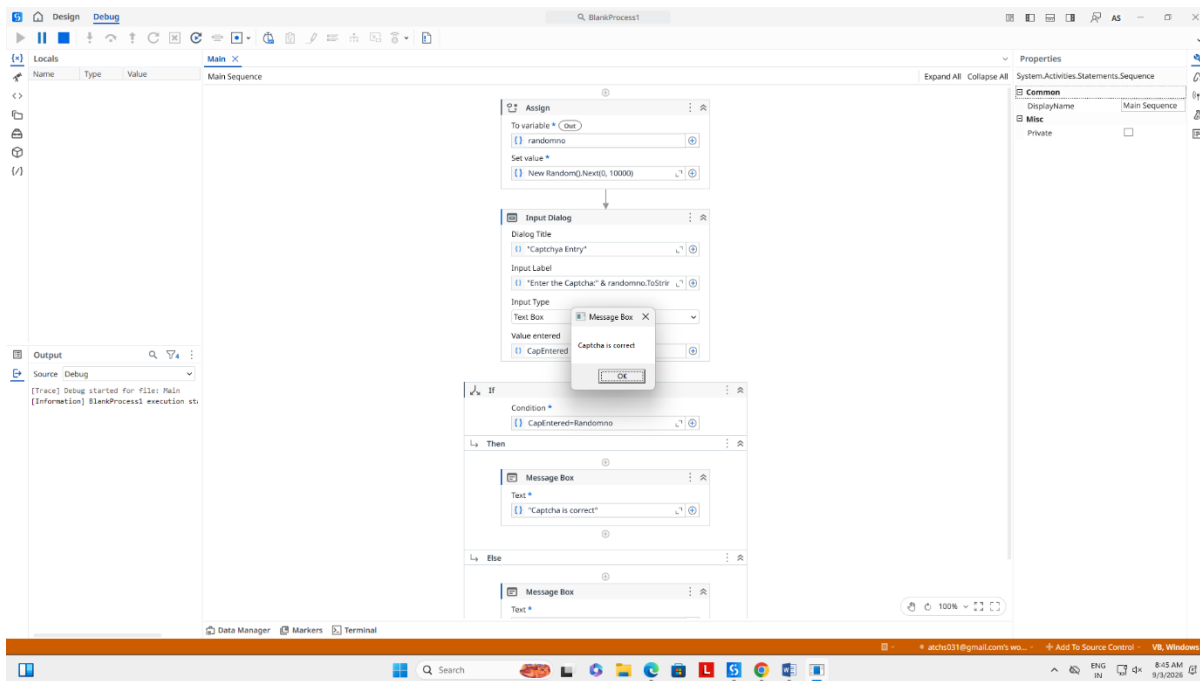
DisplayName

Main Sequence

Misc

Private

OUTPUT



RESULT:

Thus, the program has been executed successfully and output is verified.

5. Design a simple Bot for reading Email using UiPath.

AIM:

To design a simple bot for reading Email using UiPath.

ALGORITHM:

- Step 1: Start the UiPath automation process and create a new workflow.
- Step 2: Add a Get Newest Email activity and configure the Gmail account and Inbox folder.
- Step 3: Create a variable named emails with the GmailMessage data type.
- Step 4: Configure the Get Newest Email activity to retrieve the latest email from the Inbox.
- Step 5: Store the retrieved email in the emails variable using the Output Email property.
- Step 6: Add a Message Box activity to display the details of the retrieved email.
- Step 7: Use the From.Address property to display the sender's email address.
- Step 8: Use the Subject and Body properties along with Environment.NewLine to display the email subject and message content.
- Step 9: Run the workflow and verify that the latest email details are displayed correctly in the Message Box.

WORKFLOW:

Data Manager

Output

Markers

Terminal

Errors

{x}

Variables

{a}

Arguments

Namespaces

Connections

Name	Data Type	Scope	Default Value
Create variable			
{x} emails	GmailMessage	Main	{}

Main

Main Sequence

Expand All

Collapse All

Get Newest Email

Gmail

nandhudm2004@gmail.com

Email folder or label *

Inbox

Additional filters

Click to add filter

Message Box

Text *

{ } "From: " + emails.From.Address + Enviro

Properties

UIPath.GSuite.Activities.GetNewestEmailConn...

Options

Unread only

Yes

No

With attachments only

Yes

No

Important only

Yes

No

Starred only

Yes

No

Mark as read

Yes

No

Output

Email (Out)

{ } emails

Expression Editor

Message Box > Text (InArgument<Object>)

The message to display

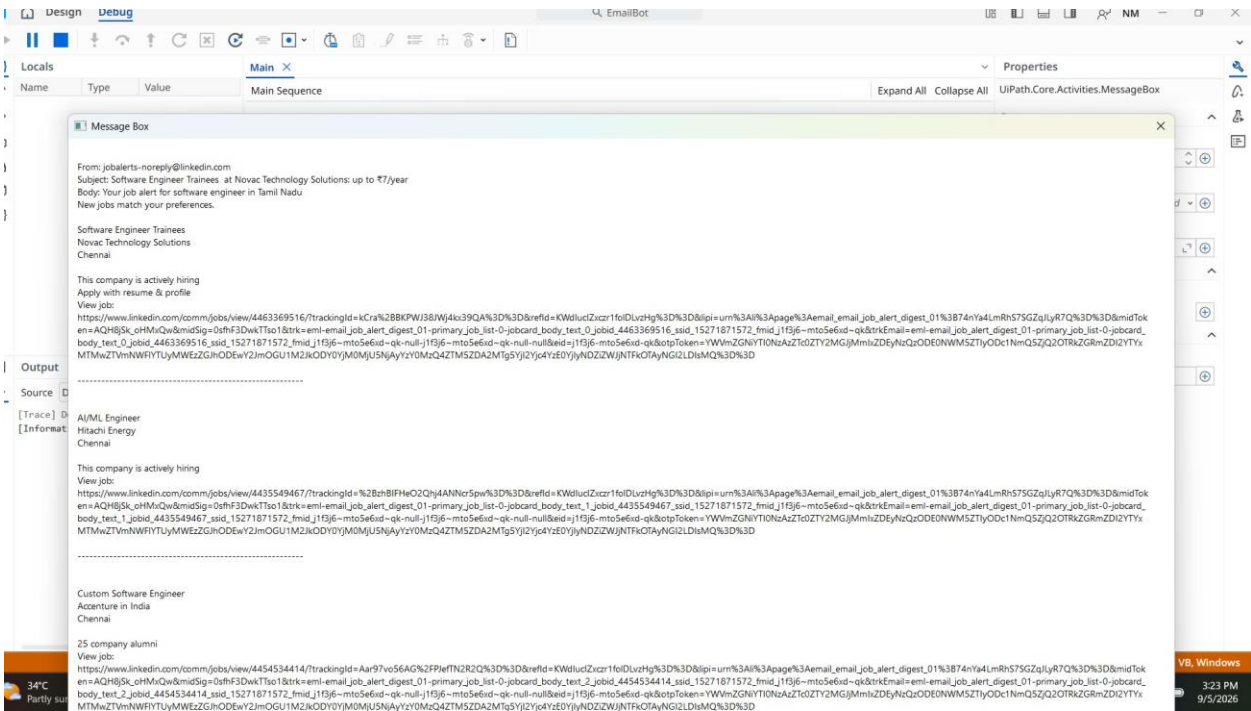
1 "From: " + emails.From.Address + Environment.NewLine + "Subject: " + emails.Subject + Environment.NewLine + "Body: " + emails.Body

Describe your expression in natural language...

Save

Cancel

OUTPUT:



RESULT:

Thus, the program has been executed successfully and output is verified.

6. Develop a Bot that extracts data from a webpage and saves it into a structured format such as Excel.

AIM:

To develop a bot that extracts data from a webpage and saves it into a structured format such as Excel.

ALGORITHM:

Step 1: Start the UiPath automation process and create a new workflow.

Step 2: Add a Chrome HTML Tables activity and enter the URL of the required webpage.

Step 3: Configure the Extract Table Data activity to select and extract the required table from the webpage.

Step 4: Create a variable named extractedData with the DataTable data type.

Step 5: Store the extracted webpage table in the extractedData variable.

Step 6: Add a Use Excel File activity and specify the Excel workbook path where the extracted data will be saved.

Step 7: Add a Write DataTable to Excel activity inside the Use Excel File activity.

Step 8: Set extractedData as the data to write and specify Excel.Sheet("Sheet1").Range("A1") as the destination.

Step 9: Run the workflow and verify that the webpage data is extracted and saved correctly in the Excel file.

WORKFLOW:

RPA Pro 6

Design Debug

Main Sequence

Chrome HTML Tables

URL: `*https://www.w3schools.com/html/html`

Open URL in a new tab

Do

Extract Table Data 'TABLE customers'

Use Excel File

Workbook path: `*C:\Users\sivani\OneDrive\Documents`

Reference as: Excel

Create if not exists: Yes

Save changes: Yes

Properties

Activity is referencing an element. Target properties should be edited from Object Explorer. [Unlink](#) [Show](#)

UIPath.UIAutomationNext.Activities.NExtractD...

Number of items: 0

Delay between pages: Default is 1.5

Generate data type: No

Input mode: Default is Same as App/Browser

Healing Agent mode: Same as App/Browser

Continue on error: Yes

Timings

Delay before: Default is 0.2 seconds

Delay after: Default is 0.3 seconds

Timeout: Default is 30 seconds

Input/Output

Destination data table: Out

Input element: Enter a VB expression

Input DataTable: Enter a VB expression

Data Manager

Output

Markers

Terminal

Errors

Variables

Arguments

Namespaces

Connections

Name	Data Type	Scope	Default Value
extractedData	DataTable	Chrome HTML Tables	

29°C Sunny

11:22 05-09-2026

RPA Pro 6

Design Debug

Main Sequence

Use Excel File

Workbook path: `*C:\Users\sivani\OneDrive\Documents`

Reference as: Excel

Create if not exists: Yes

Save changes: Yes

Do

Write DataTable to Excel

What to write: extractedData

Destination: Excel.Sheet("Sheet1").Range("A1")

Append: No

Exclude headers: No

Properties

Activity is referencing an element. Target properties should be edited from Object Explorer. [Unlink](#) [Show](#)

UIPath.UIAutomationNext.Activities.NExtractD...

Number of items: 0

Delay between pages: Default is 1.5

Generate data type: No

Input mode: Default is Same as App/Browser

Healing Agent mode: Same as App/Browser

Continue on error: Yes

Timings

Delay before: seconds

Delay after: seconds

Timeout: Default is 30 seconds

Input/Output

Destination data table: Out

Input element: Enter a VB expression

Input DataTable: Enter a VB expression

Data Manager

Output

Markers

Terminal

Errors

Variables

Arguments

Namespaces

Connections

Name	Data Type	Scope	Default Value
extractedData	DataTable	Chrome HTML Tables	

Type: Sequence

Name: Do

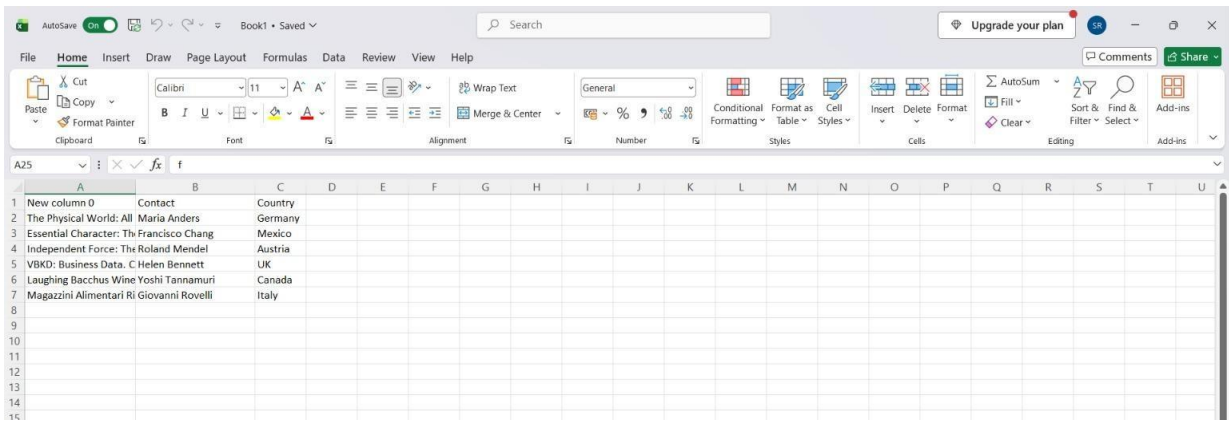
Executes a set of child activities according to a single, defined order.

Included in Default Activities

29°C Sunny

11:23 05-09-2026

OUTPUT:



The screenshot shows the Microsoft Excel interface with a table containing 7 rows and 3 columns. The columns are labeled 'New column 0', 'Contact', and 'Country'. The data rows are as follows:

	New column 0	Contact	Country
1			
2	The Physical World: All	Maria Anders	Germany
3	Essential Character: Th	Francisco Chang	Mexico
4	Independent Force: The	Roland Mendel	Austria
5	VBKD: Business Data. C	Helen Bennett	UK
6	Laughing Bacchus Wine	Yoshi Tannamuri	Canada
7	Magazzini Alimentari Ri	Giovanni Rovelli	Italy

RESULT:

Thus, the program has been executed successfully and output is verified.

7. Develop a Bot to find movie rating.

AIM:

To develop a bot to find movie rating.

ALGORITHM:

Step 1: Start the UiPath automation process and create a new workflow.

Step 2: Create two variables named movieName and rating with the String data type.

Step 3: Add an Input Dialog activity to ask the user to enter the movie name.

Step 4: Store the movie name entered by the user in the movieName variable.

Step 5: Add a Use Application/Browser activity and open the IMDb website.

Step 6: Add a Type Into activity and enter the movieName in the IMDb search box.

Step 7: Add a Click activity to select the required movie from the search results.

Step 8: Add a Get Text activity to extract the movie rating and store it in the rating variable.

Step 9: Add a Message Box activity to display the movie name and its rating, then run the workflow and verify the result.

WORKFLOW:

Data Manager

Output

Markers

Terminal

Errors

{x} Variables

{a} Arguments

Namespaces

Connections

Name

Q

Data Type

Scope

↕

Default Value

Create variable

{x} movieName

String

Main

{}

{x} rating

String

Main

{}

Main * X

Main Sequence

Expand All Collapse All

+

Input Dialog

Dialog Title

{ } "Movie Rating" L¹ +

Input Label

{ } "Enter Movie Name" L¹ +

Input Type

Text Box

Value entered

{ } movieName +

↓

Chrome IMDb Ratings, Reviews, and Where to Wa

URL

Open URL in a new tab

{ } "https://www.imdb.com/" L¹ +

Main * X

Main Sequence

Expand All Collapse All

U

movieName

+

↓

Chrome IMDb Ratings, Reviews, and Where to Wa

URL

Open URL in a new tab

{ } "https://www.imdb.com/" L¹ +

↳ Do

Tr

Type Into 'All'

Type this *

{ } movieName L¹ +

Verify

Auto

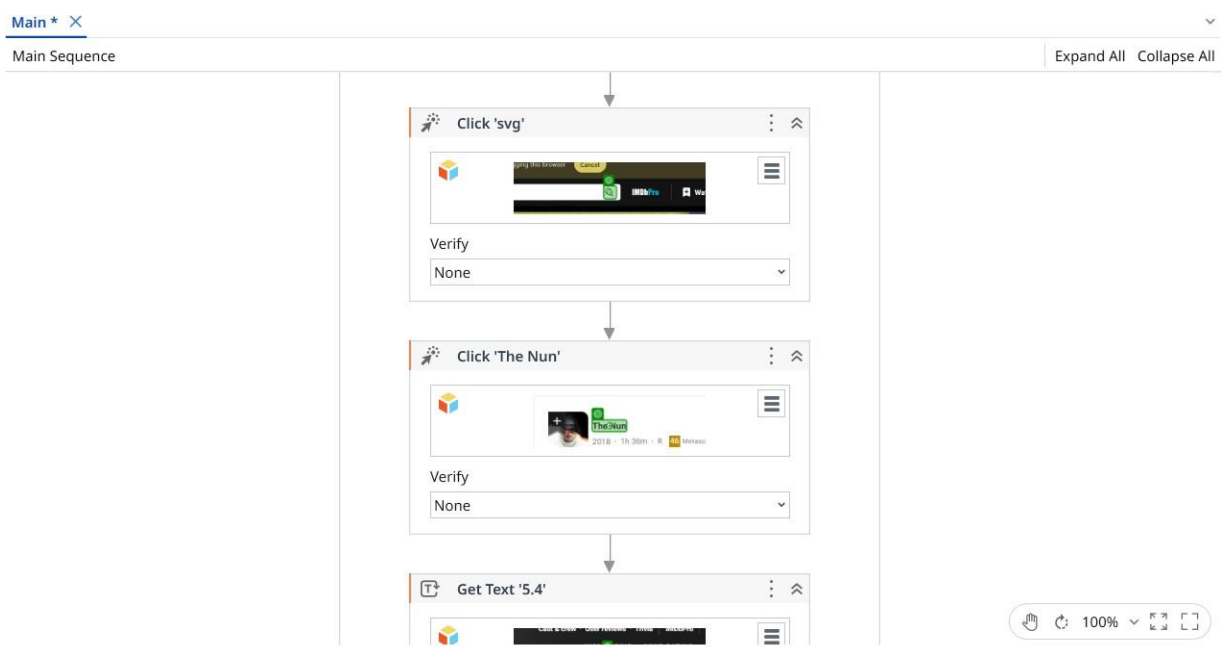
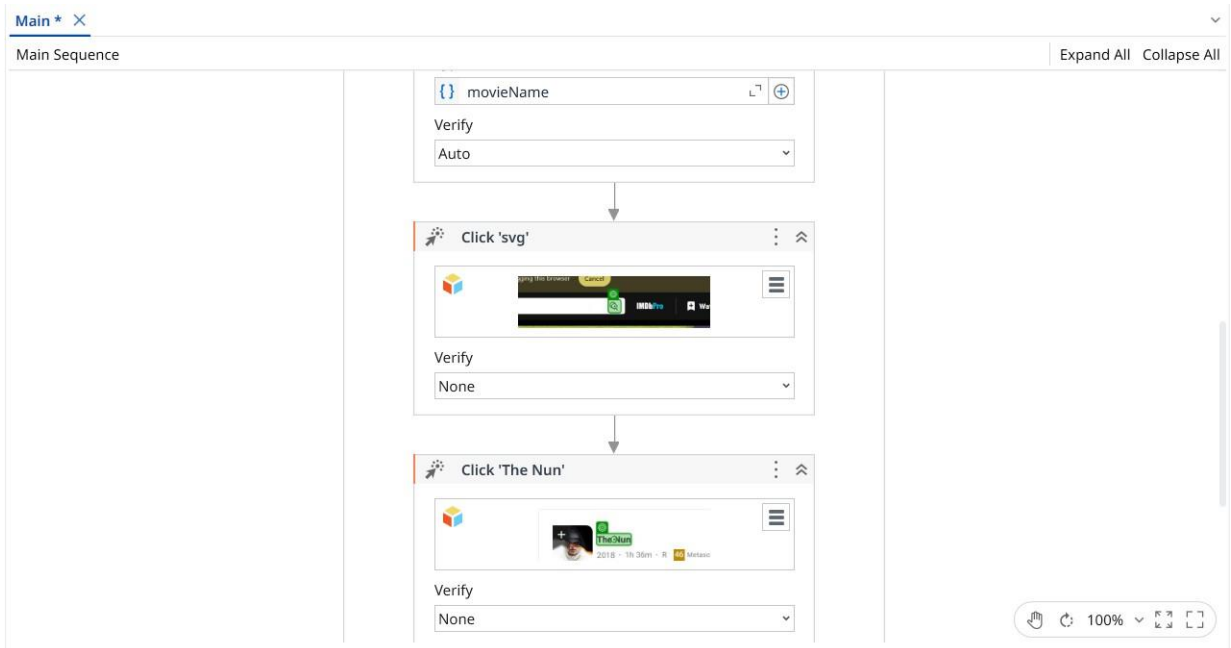
🖱️

🔄

100%

🔍

📐



Main * X

Main Sequence

Expand All Collapse All

Verify

None

Get Text '5.4'

Message Box

Text *

{ } "Movie: " + movieName + Environment.N

100%

Properties

Activity is referencing an element. Target properties should be edited from Obje...
[Unlink](#) [Show](#)

UIPath.UIAutomationNext.Activities.NGetText

☐ Text formatting

☐ Ignore hidden text

☐ Word coordinates

Healing Agent mode

Same as App/Browser

Continue on error

☒ No

Timings

Delay before

{ } Default is 0.2 seconds

Delay after

{ } Default is 0.3 seconds

Timeout

{ } Default is 30 seconds

Input/Output

Text value [Out](#)

{ } rating

Words info [Out](#)

{ } WordsInfo

Input element

{ } Enter a VB expression

Output element [Out](#)

{ } OutUIElement

Data Manager

Output

Markers

Terminal

Errors

{*} Variables

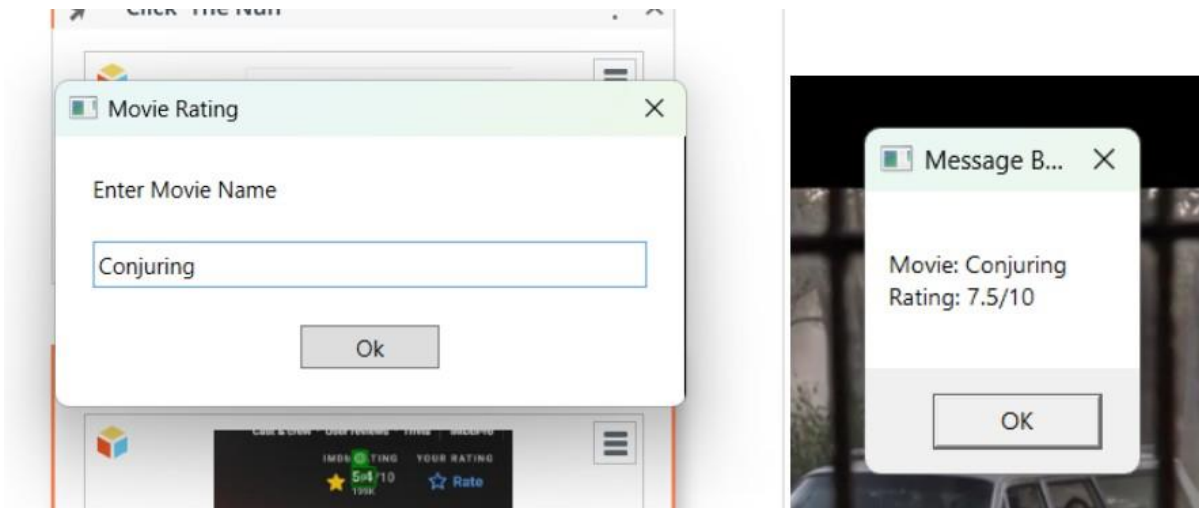
{a} Arguments

Namespaces

Connections

Name	Data Type	Scope	Default Value
Create variable			
{*} movieName	String	Main	{ }
{*} rating	String	Main	{ }

OUTPUT:



RESULT:

Thus, the program has been executed successfully and output is verified.

8. Design a Bot to create and deliver the invoice.

AIM:

To design a bot to create and deliver the invoice.

ALGORITHM:

Step 1: Start the UiPath automation process and create a new workflow.

Step 2: Create two variables named movieName and rating with the String data type.

Step 3: Add an Input Dialog activity to ask the user to enter the movie name.

Step 4: Store the movie name entered by the user in the movieName variable.

Step 5: Add a Use Application/Browser activity and open the IMDb website.

Step 6: Add a Type Into activity and enter the movieName in the IMDb search box.

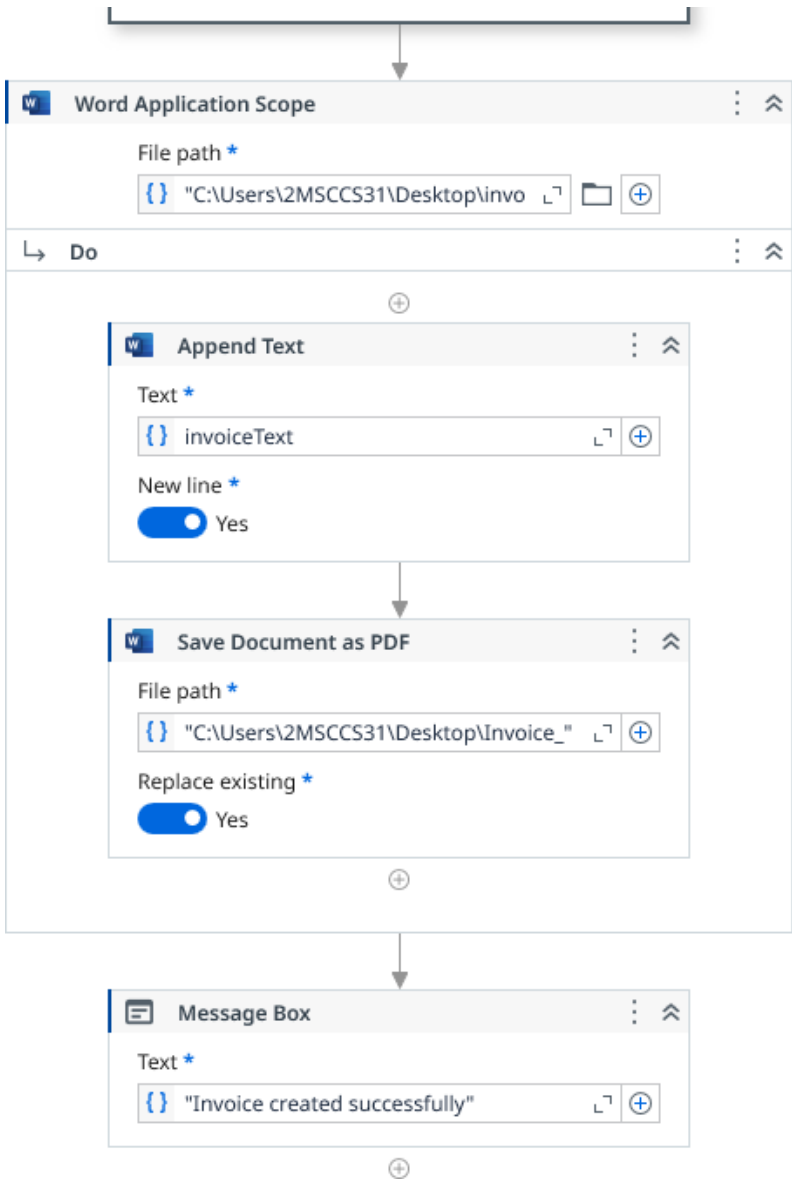
Step 7: Add a Click activity to select the required movie from the search results.

Step 8: Use Append Text to add the invoice details and Save Document as PDF to create the invoice PDF.

Step 9: Use a Message Box to display the invoice creation success message and verify the PDF.

WORKFLOW:

Data Manager	Output	Markers	Terminal	Errors
Variables	Arguments	Namespaces	Connections	
Name	Data Type	Scope		Default Value
{*} total	Double	Main		()
{*} price	Double	Main		()
{*} quantity	Int32	Main		()
{*} productName	String	Main		()



Assign

To variable * **Out**

{ } total

Set value *

{ } quantity * price

Assign

To variable * **Out**

{ } invoiceText

Set value *

{ } "=====

Word Application Scope

File path *

{ } "C:\Users\2MSCCS31\Desktop\invo

Do

Append Text

Text *

{ } invoiceText

New line *

☒ Yes

↓

Input Dialog

Dialog Title

{ } "Product Details" L⁷ +

Input Label

{ } "Enter Quantity" L⁷ +

Input Type

Text Box ▾

Value entered

{ } quantity +

↓

Input Dialog

Dialog Title

{ } "Product Details" L⁷ +

Input Label

{ } "Enter Price Per Unit" L⁷ +

Input Type

Text Box ▾

Value entered

{ } price +

↓

Assign

To variable *

Out

{ } total +

Set value *

Input Dialog

Dialog Title
{} "Invoice Details" L⁷ +

Input Label
{} "Enter Invoice Number" L⁷ +

Input Type
Text Box v

Value entered
{} invoiceNumber +

Input Dialog

Dialog Title
{} "Product Details" L⁷ +

Input Label
{} "Enter Product Name" L⁷ +

Input Type
Text Box v

Value entered
{} productName +

Input Dialog

Dialog Title
{} "Product Details" L⁷ +

+

Input Dialog

Dialog Title

{ }

"Invoice Details"

↵

+

Input Label

{ }

"Enter Customer Name"

↵

+

Input Type

Text Box

▼

Value entered

{ }

customerName

+

+

Input Dialog

Dialog Title

{ }

"Invoice Details"

↵

+

Input Label

{ }

"Enter Customer Email"

↵

+

Input Type

Text Box

▼

Value entered

{ }

customerEmail

+

+

Input Dialog

Dialog Title

{ }

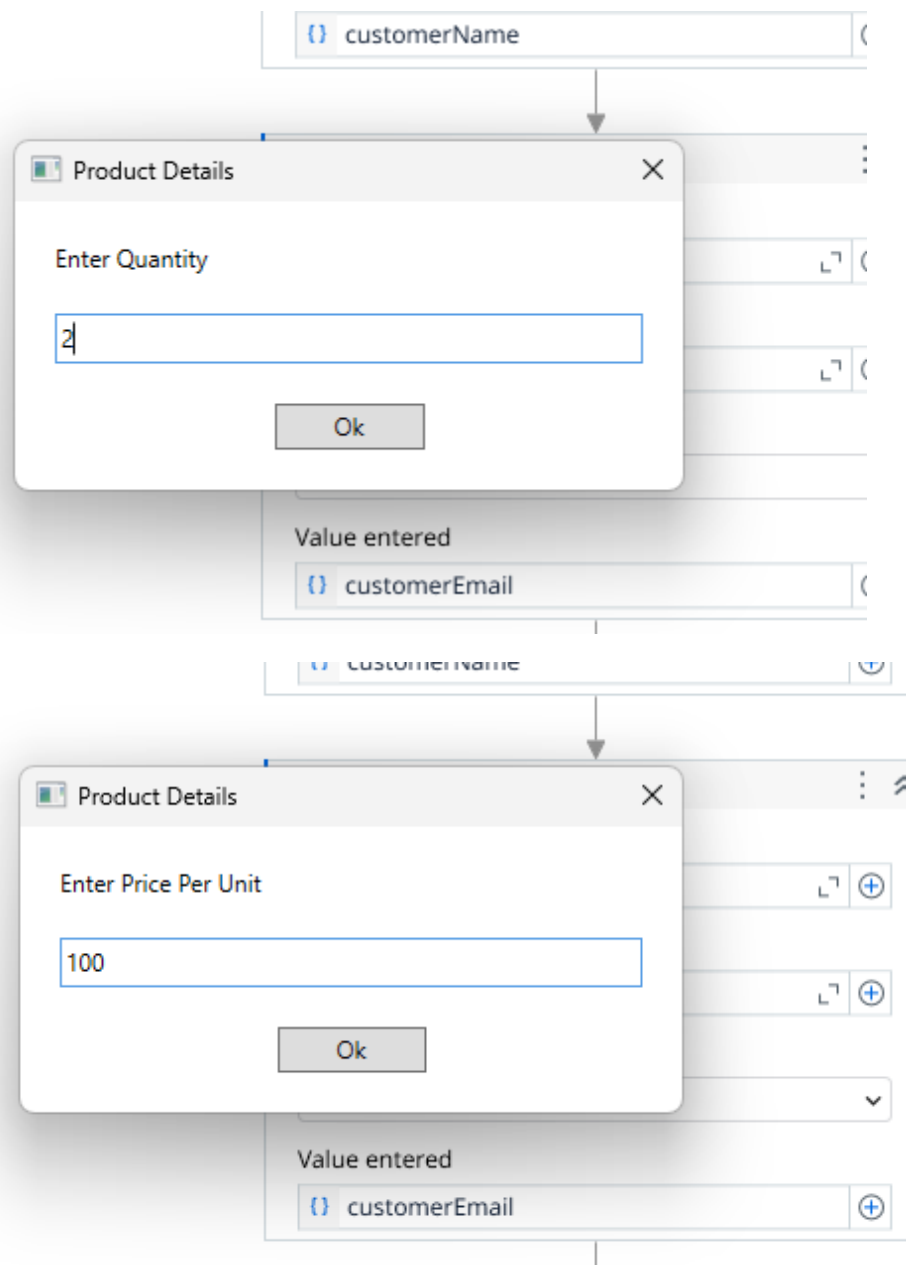
"Invoice Details"

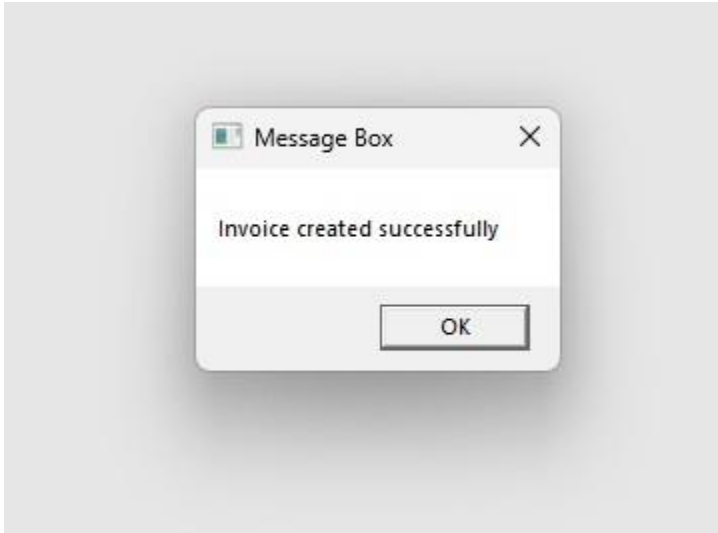
↵

+

Input Label

OUTPUT:





=====

INVOICE

Invoice Number: INV001

Customer Name: Nandhitha

Customer Email: nandhu2004@gmail.com

Product: Butter

Quantity: 2

Price: ₹100

Total: ₹200

Grand Total: ₹200

=====

Product Details

Enter Product Name

Ok

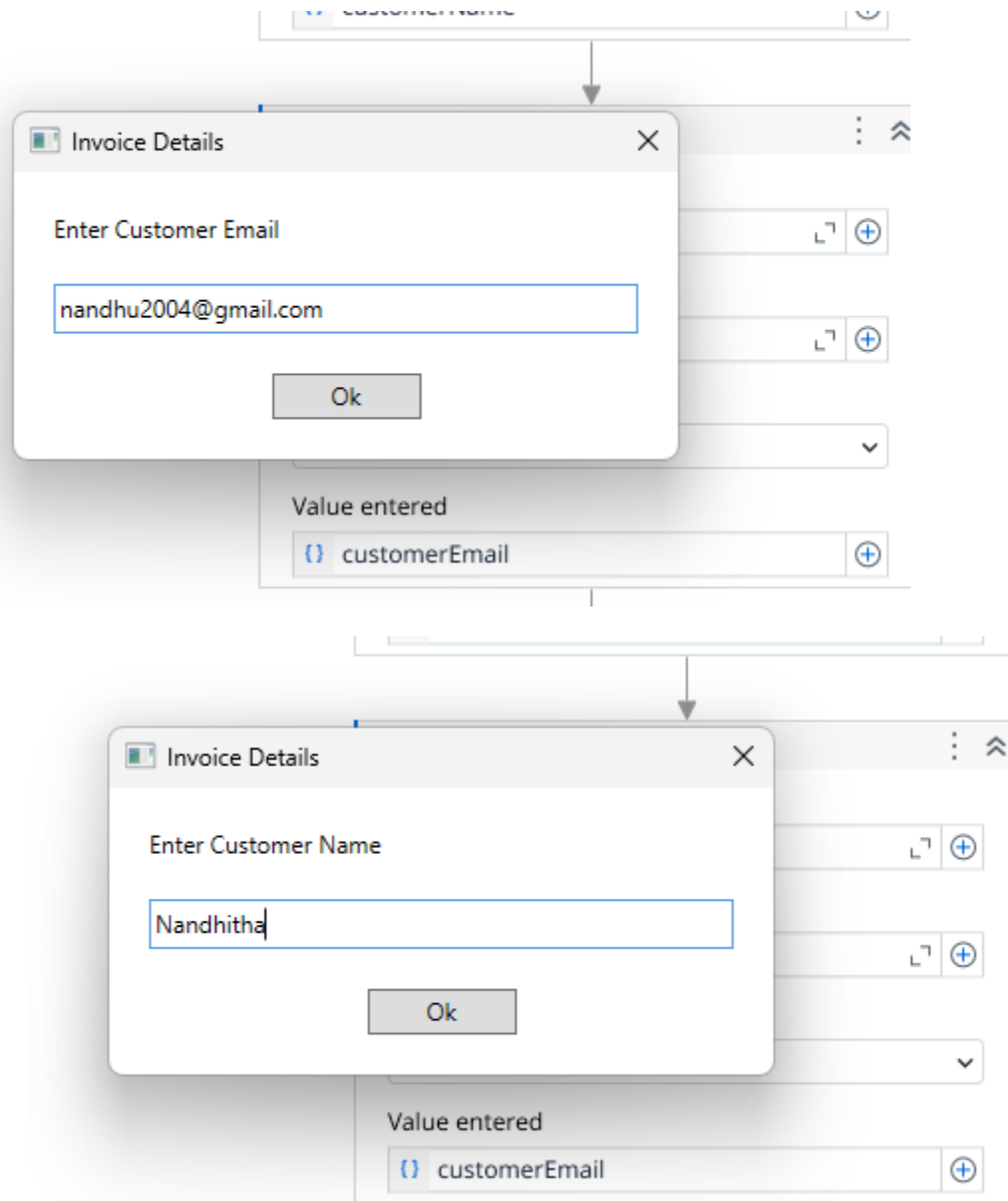
Value entered
{ } customerEmail

Invoice Details

Enter Invoice Number

Ok

Value entered
{ } customerEmail



RESULT:

Thus, the program has been executed successfully and output is verified.

